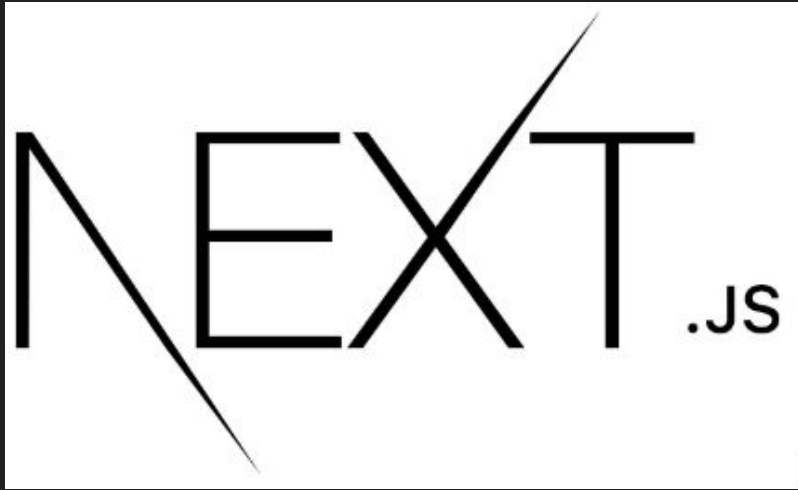


Montando uma Aplicação utilizando NextJS e FireBase



e fazendo deploy com a Vercel



<https://aula-firebase.vercel.app/>

1. Introdução e História:



Breve história do Firebase e Firestore:

- Firebase foi fundado em 2011 como uma startup independente, focada em DX, oferecendo uma plataforma de desenvolvimento de aplicativos móveis.
- Em 2014, o Firebase foi adquirido pelo Google.
- Firestore foi lançado em 2019 como parte da plataforma Firebase, oferecendo um banco de dados NoSQL em tempo real e escalável para aplicativos web e móveis.

Motivação por trás do desenvolvimento:

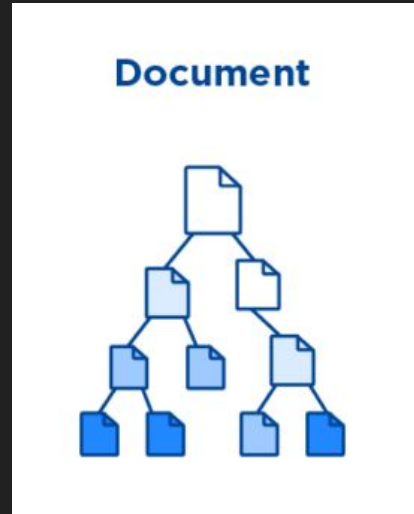


- A necessidade de um banco de dados flexível, escalável e em tempo real para suportar aplicativos modernos que lidam com grandes volumes de dados e usuários simultâneos.
- Simplificar o desenvolvimento de aplicativos, servindo como BaaS (Backend as a Service), fornecendo uma plataforma abrangente que inclui banco de dados, autenticação, recuperação de senha, hospedagem, mensagens, análises, entre outros recursos.

2. Características Principais:

Estrutura de dados e modelo de armazenamento:

- Firestore utiliza uma estrutura de dados baseada em coleções e documentos.
- Os documentos são armazenados em coleções e podem conter campos de dados, como strings, números, booleanos, arrays e até mesmo subcoleções.
- Os dados são organizados em uma estrutura hierárquica, permitindo consultas eficientes e escalabilidade.



Linguagem de consulta:



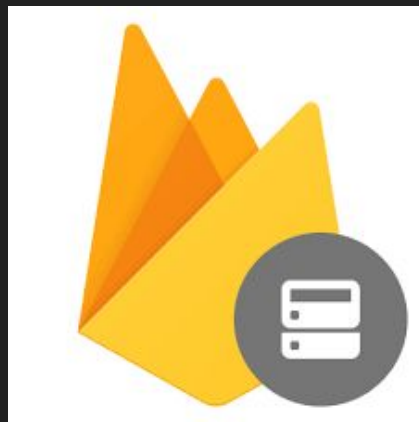
- Firestore oferece uma linguagem de consulta baseada em SQL, permitindo que os desenvolvedores realizem consultas complexas para recuperar dados de maneira eficiente.

```
import { collection, query, where, getDocs } from "firebase/firestore";

const q = query(collection(db, "cities"), where("capital", "==", true));

const querySnapshot = await getDocs(q);
querySnapshot.forEach((doc) => {
  // doc.data() is never undefined for query doc snapshots
  console.log(doc.id, " => ", doc.data());
});
```

3. Vantagens e Desvantagens:



Principais vantagens:

- **Tempo real:** Os dados são sincronizados automaticamente entre os clientes e o servidor, permitindo atualizações em tempo real.
- **Escalabilidade:** Firestore é altamente escalável, podendo suportar desde pequenos aplicativos até grandes volumes de dados e usuários.
- **Facilidade de uso:** Integração direta com outros serviços Firebase e uma API simples de usar tornam o desenvolvimento mais fácil e rápido.
- **Baixo Custo Inicial:** O Firebase tem um modelo de preços flexível que permite começar com custo zero. Muitos dos serviços do Firebase oferecem uma camada gratuita generosa.

- **Limitações e desafios:**



- **Custo:** Para grandes volumes de dados e tráfego, os custos podem se tornar significativos.
- **Complexidade de consultas:** Consultas complexas podem exigir planejamento cuidadoso para garantir um desempenho adequado.

4. Casos de Uso Típicos:



Aplicações e cenários:

- Aplicativos em tempo real, como aplicativos de mensagens, redes sociais e colaboração em tempo real.
- Aplicativos que exigem sincronização de dados entre dispositivos e plataformas.
- Aplicativos que precisam lidar com grandes volumes de dados e usuários simultâneos.

5. Comparação com Outros Tipos de Bancos de Dados NoSQL:



Comparação:

- Em comparação com outros bancos de dados NoSQL, como MongoDB ou Couchbase, Firestore se destaca pela sua sincronização em tempo real e integração direta com outros serviços Firebase.
- Ao escolher entre diferentes bancos de dados NoSQL, é importante considerar os requisitos específicos do projeto, como escalabilidade, necessidades de consulta e integração com outros serviços. Para aplicativos que exigem sincronização em tempo real e uma ampla gama de serviços, Firestore pode ser uma escolha ideal.

6. Exemplos de Empresas ou Projetos:

Exemplos de empresas ou projetos:

- Google utiliza Firestore em vários de seus próprios produtos, como Google Photos e Google Analytics.
- Strava, um aplicativo popular de rastreamento de atividades físicas, utiliza Firestore para sincronização de dados em tempo real.
- Whatsapp, Instagram, Google Docs, Duolingo, Uber, são exemplos de aplicativos escaláveis que utilizam o Firebase.



Google Analytics

Uber



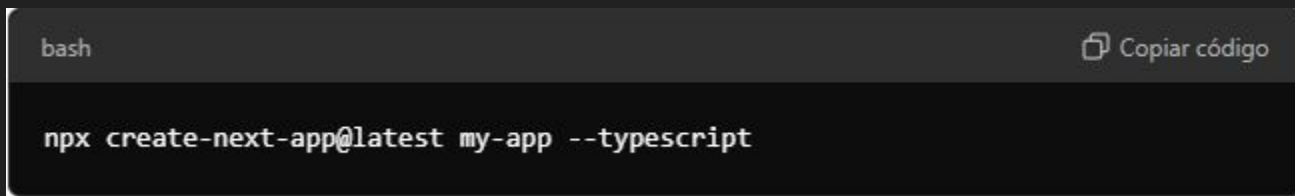
Google Docs



HANDS ON FRONT-END

Agora vamos criar uma aplicação usando NextJS

```
npx create-next-app@latest meuApp --typescript
```

A screenshot of a terminal window with a dark background. The top bar is dark gray and contains the text 'bash' on the left and a copy icon followed by 'Copiar código' on the right. The main area of the terminal is black and displays the command 'npx create-next-app@latest my-app --typescript' in a light gray font. The cursor is positioned at the end of the command.

```
bash Copiar código  
npx create-next-app@latest my-app --typescript
```

(lembre-se de adicionar a pasta src na criação do app NextJs)

depois entre na pasta do app e digite:

```
yarn dev
```

ou

```
npx next dev
```

(caso vc não tenha yarn, <https://classic.yarnpkg.com/en/docs/install#windows-stable>)

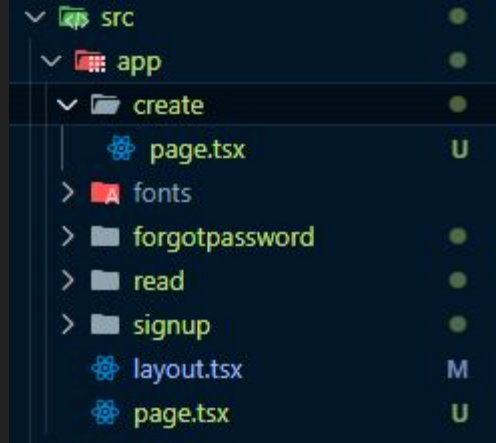
HANDS ON FRONT-END

Agora acesse <http://localhost:3000/> e voila, vamos começar

Agora abriremos o projeto no vscode (ou IDE de sua escolha)

A page.tsx vai ser a home / página inicial, usaremos ela para Login

as próximas páginas, serão criadas dentro de outras pastas, com page.tsx dentro.



CRIANDO PROJETO NO FIREBASE

Vamos criar nosso projeto no Firebase! (<https://console.firebase.google.com/>)

Entre com sua conta Google (pode ser a da faculdade mesmo)

O passo-a-passo:

Criar projeto, dar nome ao projeto, continuar, continuar....

Após isso, selecione web (`</>`) para continuarmos

Coloque o nome do app

(opcional: Firebase Hosting)

Comece adicionando o
Firebase ao seu
aplicativo



Adicione um app para começar

ADICIONANDO SDK FIREBASE

Aqui é onde adicionamos Firebase em nosso projeto

```
yarn add firebase
```

Agora vamos tratar de boas práticas de segurança

Criaremos um `.env.local` na pasta local da aplicação

E salvaremos todas as infos sensíveis dentro dele

exemplo no próximo slide

(verifique se no `.gitignore` está `.env.local`)

EXEMPLO .ENV.LOCAL

NEXT_PUBLIC_FIREBASE_API_KEY=your-api-key

NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=your-auth-domain

NEXT_PUBLIC_FIREBASE_PROJECT_ID=your-project-id

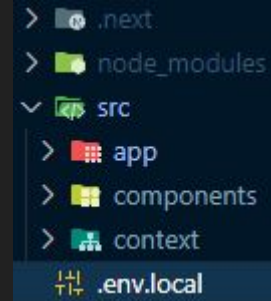
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=your-storage-bucket

NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=your-messaging-sender-id

NEXT_PUBLIC_FIREBASE_APP_ID=your-app-id

.env.local

```
1 NEXT_PUBLIC_FIREBASE_API_KEY="AIzaSyDCQWzbRe1Qlt1QYbDJvB9D4zqaF1pgdoA"
2 NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN="my-app-8cdad.firebaseio.com"
3 NEXT_PUBLIC_FIREBASE_PROJECT_ID="my-app-8cdad"
4 NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET="my-app-8cdad.appspot.com"
5 NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID="409571712062"
6 NEXT_PUBLIC_FIREBASE_APP_ID="1:409571712062:web:3515c27b6d8520b841856f"
7 NEXT_PUBLIC_FIREBASE_MEASUREMENT_ID="G-CMX4RE1E2N"
```



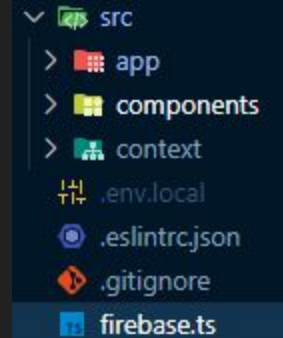
CRIAR ARQUIVO firebase.ts

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore";
import { getStorage } from "firebase/storage";

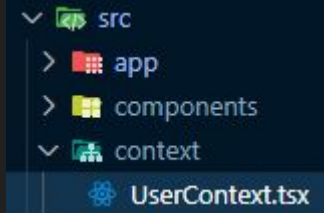
const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,
};

const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
const db = getFirestore(app);
const storage = getStorage(app);

export { app, auth, db, storage };
```



HANDS ON REACT CONTEXT



Agora vamos criar Context, e o que é isso afinal?

Context é para manter o contexto do usuário pela aplicação toda.

No nosso caso, manteremos o e-mail de login durante a aplicação.

Primeiro vamos criar a pasta context, dentro de src

Criaremos o arquivo UserContext.tsx

REACT CONTEXT

Bom, aqui dá uma complicada na vida, pois estamos usando TypeScript

Qual a idéia do TS? Basicamente um JS tipado.

A gente sempre tem que identificar o tipo dos objetos, e podemos criar interfaces para facilitar e reutilizar o código.

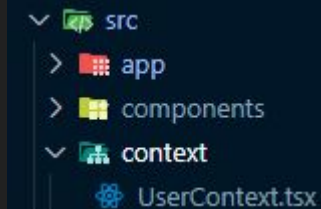
Para a autenticação, vamos utilizar da doc do Firebase com umas colinhas de ReactJS

(<https://firebase.google.com/docs/auth/web/password-auth?hl=pt-br>)

```

/** @format */
"use client"; // Ensure this file is treated as a Client Component
import React, { createContext, useContext, useEffect, useState } from "react";
import { onAuthStateChanged, User } from "firebase/auth";
import { auth } from "../../firebase";
interface UserContextType {
  user: User | null;
  loading: boolean;
}
const UserContext = createContext<UserContextType | undefined>(undefined);
export const UserProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (currentUser) => {
      setUser(currentUser);
      setLoading(false);
    });
    return () => unsubscribe(); // Clean up the subscription
  }, []);
  return <UserContext.Provider value={{ user, loading }}>{!loading && children}</UserContext.Provider>;
};
export const useUser = () => {
  const context = useContext(UserContext);
  if (context === undefined) {
    throw new Error("useUser must be used within a UserProvider");
  }
  return context;
};

```



ADICIONANDO CONTEXT NA APLICAÇÃO

Agora vamos adicionar o contexto na aplicação.

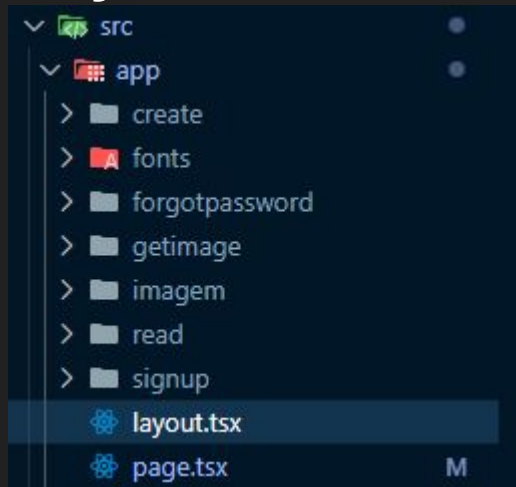
Entraremos no arquivo `src/app/layout.tsx`

e adicionaremos o `UserProvider` envolvendo o `{children}`

```
/** @format */
import { UserProvider } from "@context/UserContext";
import React from "react";

export const metadata = {
  title: "App do V",
  description: "Aqueeeeele APP",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html Lang="en">
      <body>
        <UserProvider>{children}</UserProvider>
      </body>
    </html>
  );
}
```



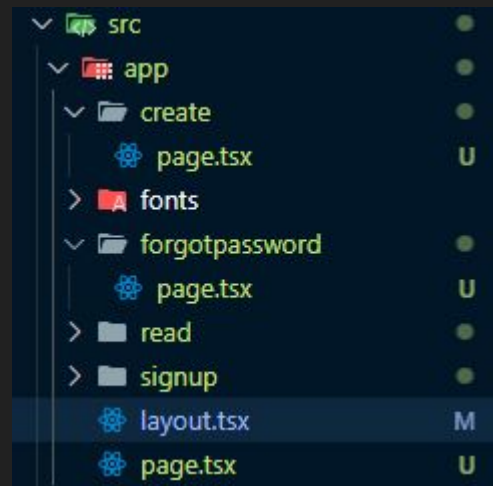
HANDS ON FRONT-END

Agora vamos mexer na nossa index.tsx, e fazer dela nossa login page.

Bom, para utilizar todas as funcionalidades do FireBase Authentication, vamos fazer um frontend com novo usuário e com esqueceu a senha.

Basicamente será um front-end com um form, dois inputs, dois links e um botão
nos próximo slide tem o código para agilizar o processo.

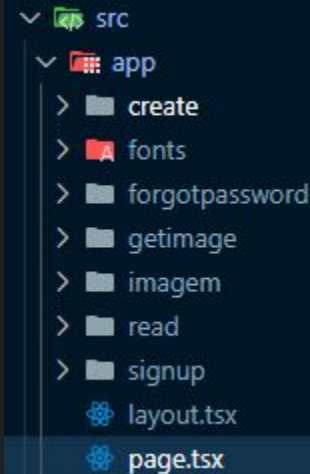
Já vamos criar as pastas signup, forgotpassword, create
Dentro de cada, crie um page.tsx



```

/* @format */
// src/app/page.tsx
"use client" // Add this line to mark the component as a Client Component
import { useState } from "react";
import { useRouter } from "next/navigation"; // Change this import to next/navigation
import { signInWithEmailAndPassword } from "firebase/auth";
import Link from "next/link";
import { auth } from "../../firebase";
const HomePage = () => {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const router = useRouter();
  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      await signInWithEmailAndPassword(auth, email, password);
      router.push("/create"); // Redirect to a protected route after login
    } catch (error) {
      console.error("Error signing in: ", error);
    }
  }
  return (
    <div style={ { display: "flex", justifyContent: "center", alignItems: "center", height: "100vh", flexDirection: "column" } }>
      <h1 style={ { marginBottom: "20px" } }>8em windol </h1>
      <form
        onSubmit={handleSubmit}
        style={ { display: "flex", flexDirection: "column", width: "300px" } }
      >
        <input
          type="email"
          value={email}
          onChange={ (e) => setEmail(e.target.value) }
          placeholder="Email"
          required
          style={ { marginBottom: "10px", padding: "8px", borderRadius: "4px", border: "1px solid #ccc" } }
        />
        <input
          type="password"
          value={password}
          onChange={ (e) => setPassword(e.target.value) }
          placeholder="Password"
          required
          style={ { marginBottom: "10px", padding: "8px", borderRadius: "4px", border: "1px solid #ccc" } }
        />
        <button
          type="submit"
          style={ { padding: "10px", backgroundColor: "blue", color: "white", border: "none", borderRadius: "4px", cursor: "pointer" } }
        >
          Entrar
        </button>
      </form>
      <div style={ { marginTop: "20px", display: "flex", justifyContent: "space-between", width: "300px" } }>
        <Link
          href="/signup"
          style={ { color: "blue", textDecoration: "underline", cursor: "pointer" } }
        >
          Cadastrar e-mail
        </Link>
        <Link
          href="/forgotpassword"
          style={ { color: "blue", textDecoration: "underline", cursor: "pointer" } }
        >
          Esqueceu a senha?
        </Link>
      </div>
    </div>
  );
}
export default HomePage;

```



```

/** @format */
"use client"; // Ensure this file is treated as a Client Component
import { useState } from "react";
import { useRouter } from "next/navigation"; // Change the router import to next/navigation
import { createUserWithEmailAndPassword } from "firebase/auth";
import { auth } from "../../firebase"; // Adjust the path based on your structure
import Link from "next/link";

export default function Signup() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const router = useRouter();

  const handleSignup = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(""); // Clear any previous errors
    try {
      await createUserWithEmailAndPassword(auth, email, password);
      // Redirect to the dashboard or homepage after successful signup
      router.push("/"); // Adjust this path based on your app structure
    } catch {
      // Handle signup error
      setError("Failed to create an account. Please check your email and password. ");
    }
  };

  return (
    <div style={{ display: "flex", justifyContent: "center", alignItems: "center", height: "100vh" }}>
      <form
        onSubmit={handleSignup}
        style={{ display: "flex", flexDirection: "column", width: "300px" }}
      >
        <h2>Cadastrar</h2>
        <input
          type="email"
          placeholder="Email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
          style={{ marginBottom: "10px", padding: "8px" }}
        />
        <input
          type="password"
          placeholder="Password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
          style={{ marginBottom: "10px", padding: "8px" }}
        />
        {error ? <p style={{ color: "red" }}>{error}</p> : <button
          type="submit"
          style={{ padding: "8px", backgroundColor: "blue", color: "white", cursor: "pointer" }}
        >
          Cadastre-se
        </button>
        <div style={{ marginTop: "20px", display: "flex", justifyContent: "space-between" }}>
          <Link
            href="/"
            style={{ color: "blue", textDecoration: "underline", cursor: "pointer" }}
          >
            Voltar para Login
          </Link>
        </div>
      </form>
    </div>
  );
}

```

src

app

- > create
- > fonts
- > forgotpassword
- > getimage
- > imagem
- > read
- > signup

page.tsx

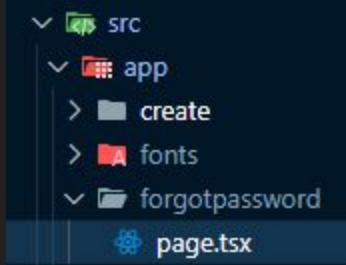
```

/** @format */
"use client"; // Ensure this file is treated as a Client Component
import { useState } from "react";
import { useRouter } from "next/navigation"; // Import from next/navigation
import { sendPasswordResetEmail } from "firebase/auth";
import { auth } from "../../firebase"; // Adjust the path based on your structure
import Link from "next/link";

export default function ForgotPassword() {
  const [email, setEmail] = useState("");
  const [message, setMessage] = useState("");
  const [error, setError] = useState("");
  const router = useRouter(); // Use useRouter from next/navigation
  const handleResetPassword = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(""); // Clear any previous errors
    setMessage(""); // Clear any previous messages
    try {
      await sendPasswordResetEmail(auth, email);
      setMessage("Check your inbox for further instructions to reset your password. ");
      setEmail(""); // Clear email input after sending the reset email
      router.push("/"); // Redirect to the login page
    } catch {
      setError("Failed to send password reset email. Please check your email address. ");
    }
  };

  return (
    <div style={{ display: "flex", justifyContent: "center", alignItems: "center", height: "100vh", flexDirection: "column" }}>
      <form
        onSubmit={handleResetPassword}
        style={{ display: "flex", flexDirection: "column", width: "300px", padding: "20px" }}
      >
        <h2 style={{ marginBottom: "20px" }}>Esqueceu a senha? </h2>
        <input
          type="email"
          placeholder="Email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
          style={{ marginBottom: "10px", padding: "10px", borderRadius: "4px", border: "1px solid #ccc" }}
        />
        <error %s <p style={{ color: "red" }}>{error}</p>
        <message %s <p style={{ color: "green" }}>{message}</p>
        <button
          type="submit"
          style={{ padding: "10px", backgroundColor: "blue", color: "white", border: "none", borderRadius: "4px", cursor: "pointer" }}
        >
          Enviar e-mail para resetar
        </button>
        <div style={{ marginTop: "20px", display: "flex", justifyContent: "space-between" }}>
          <Link
            href="/"
            style={{ color: "blue", textDecoration: "underline", cursor: "pointer" }}
          >
            Voltar para Login
          </Link>
          <Link
            href="/signup"
            style={{ color: "blue", textDecoration: "underline", cursor: "pointer" }}
          >
            Novo usuário?
          </Link>
        </div>
      </form>
    </div>
  );
}

```



TESTEM AS FUNCIONALIDADES

Verifiquem se deu tudo certo, se deu coisa linda!!

Fizemos uma autenticação, uma das coisas mais chatas de fazer no backend

Agora vamos para o famoso CRUD!

Vamos criar uma página para criar, uma para ler com update e delete.

Vou colocar os códigos nos próximos slides para facilitar.

REUTILIZAR CÓDIGOS

Vamos criar um Box (modelo) para as nossas próximas páginas, para sempre ter o Header e footer em todas as páginas pós OnBoarding

```

/** @format */
import React, { ReactNode } from "react";
import { useRouter } from "next/navigation";
import { signOut } from "firebase/auth";
import { useUser } from "@context/UserContext"; // Correctly import useUser
import { auth } from "../../firebase"; // Ensure the path to firebase.ts is correct

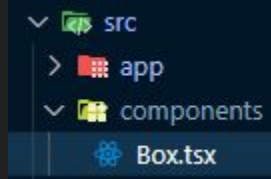
type Props = {
  children: ReactNode;
};

const Box = ({ children }: Props) => {
  const router = useRouter();
  const { user } = useUser(); // Use the custom hook directly
  const handleLogout = async () => {
    try {
      await signOut(auth); // Sign out the user
      router.push("/"); // Redirect to login page after logout
    } catch (error) {
      console.error("Logout error:", error);
    }
  };

  return (
    <div style={{ padding: "20px" }}>
      <header style={{ display: "flex", justifyContent: "space-between", alignItems: "center" }}>
        <h1>Bem vindo, {user ? user.email : "Guest"}</h1> {/* Handle user null case */}
        <button
          onClick={handleLogout}
          style={{ padding: "8px", backgroundColor: "red", color: "white" }}
        >
          Logout
        </button>
      </header>
      <main style={{ marginTop: "20px" }}>
        {children} {/* Render the child components (page content) */}
      </main>
    </div>
  );
};

export default Box;

```



CREATE PAGE

Vamos fazer a página para adicionar um dado no FireStore!

Aqui deixo para a criatividade de vocês, eu vou criar uma página com dois inputs apenas e inserir no banco de dados.

As funções são relativamente simples.

Lembrem-se sempre de dar uma olhada nos docs, tem bastante coisa em PT

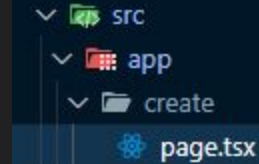
(<https://firebase.google.com/docs/firestore/manage-data/add-data?hl=pt-br>)

No próximo slide, os códigos para agilizar o processo.

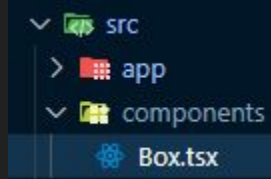
```

/** @format */
"use client ";
import { useState } from "react";
import { useRouter } from "%/context/UserContext ";
import { addDoc, collection } from "firebase/firestore "; // Import Firestore functions
import Box from "%/components/Box ";
import { db } from ".././../firebase ";
// Ensure this file is treated as a Client Component
const CreatePage = () => {
  const [ user ] = useRouter(); // Access user context
  const [ nome, setName ] = useState("");
  const [ idade, setIdade ] = useState("");
  const [ error, setError ] = useState("");
  const [ message, setMessage ] = useState("");
  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(""); // Clear any previous errors
    setName(""); // Clear previous messages
    if (!user) {
      setError("User not authenticated. ");
      return;
    }
    try {
      // Define a collection reference for users in Firestore
      const usersCollectionRef = collection(db, "users");
      // Add a new document to the collection with a generated UID
      await addDoc(usersCollectionRef, {
        nome,
        idade,
        userId: user.uid, // Optionally store the user's UID with the document
      });
      setMessage("Dados salvos!");
      setName(""); // Clear the input field
      setIdade(""); // Clear the input field
    } catch (error) {
      setError("Failed to save data. Please try again. ");
      console.error("Error saving data: ", error);
    }
  };
  return (
    <Box>
      <div style={ { padding: "20px", maxWidth: "400px", margin: "0 auto", border: "1px solid #ddd ", borderRadius: "8px", boxShadow: "0 2px 4px rgba(0, 0, 0, 0.1) ", backgroundColor: "#fff" }} >
        <h1 style={ { textAlign: "center", marginBottom: "20px", color: "#333" }} >Adicione um dado no FireStore </h1>
        <form>
          <button type="button" onClick={handleSubmit} style={ { display: "flex", flexDirection: "column" }} >
            <input type="text" placeholder="Nome" value={nome} onChange={ (e) => setName(e.target.value)} required style={ { marginBottom: "10px", padding: "10px", borderRadius: "4px", border: "1px solid #ccc ", fontSize: "16px" }} />
            <input type="number" placeholder="Idade" value={idade} onChange={ (e) => setIdade(e.target.value)} required style={ { marginBottom: "10px", padding: "10px", borderRadius: "4px", border: "1px solid #ccc ", fontSize: "16px" }} />
            <button type="submit" style={ { padding: "10px", backgroundColor: "#0070f3", color: "white", border: "none", borderRadius: "4px", cursor: "pointer ", fontSize: "16px", transition: "background-color 0.3s " }} >
              Salvar
            </button>
            <div style={ { display: "flex", justify-content: space-between, margin-top: "10px" }} >
              <div>
                <div style={ { color: "red", margin: "0" }} >{error}</div>
                <div style={ { color: "green", margin: "0" }} >{message}</div>
              </div>
            </div>
          </form>
        </div>
      </Box>
    </div>
  );
};
export default CreatePage;

```



READ PAGE



Bom agora que já adicionamos um dado e verificamos no firestore online, vamos criar uma page para pegar os dados, mostrar na tela, atualizar e deletar! os códigos estão no próximo slide para facilitar o processo.

Lembrem-se de adicionar o `<Link>` no `<header>` dentro do componente `Box.tsx`

```
<Link  
  href="/read"  
  style={{ marginRight: "20px", textDecoration: "none", color: "#0070f3" }}  
>  
  Função GET  
</Link>
```

Eu ia colocar o código aqui mas ficou muito grande.

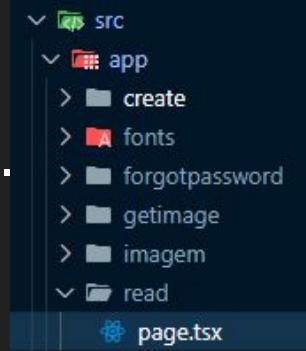
fica aqui o link do arquivo para facilitar

<https://github.com/viniciusgdoliveira/my-app/blob/main/src/app/read/page.tsx>

DOC FIRESTORE: <https://firebase.google.com/docs/firestore?hl=pt-br>

```
const handleDelete = async (id: string) => {
  try {
    const userRef = doc(db, "users", id); // Use UUID to
    await deleteDoc(userRef);
    fetchUsers(); // Refetch users after deletion
  } catch (error) {
    console.error("Error deleting user: ", error);
    setError("Failed to delete user. Please try again.");
  }
};
```

```
const handleUpdate = async (id: string) => {
  try {
    const userRef = doc(db, "users", id); // Use UUID to reference the document
    await setDoc(userRef, {
      nome: newName,
      idade: parseInt(newIdade), // Convert idade to a number
    });
    setEditingUser(null); // Close edit mode
    setNewNome(""); // Clear newName
    setNewIdade(""); // Clear newIdade
    fetchUsers(); // Refetch users to get updated data
  } catch (error) {
    console.error("Error updating user: ", error);
    setError("Failed to update user. Please try again.");
  }
};
```



FINALIZAMOS NOSSA AULA

Muito obrigado a quem ficou até aqui rererere sei que n é fácil
segue link do github do projeto para ref:

<https://github.com/viniciusgdoliveira/my-app>

EXTRA CONTENT

Bom provavelmente não deu tempo de fazer tudo, mas vou deixar aqui como referência para caso queiram fazer outrora.

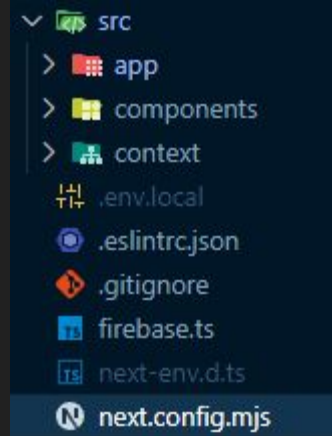
A idéia é criar uma página para upload da imagem e download

Inicialmente vamos criar mais uma pasta imagem, para colocar dentro page.tsx

Agora para conectar o NextJS com Firebase API

Vamos em next.config.mjs e adicionaremos isto

```
/** @type {import('next').NextConfig} */  
const nextConfig = {  
  images: {  
    domains: ['firebasestorage.googleapis.com'],  
  },  
};  
  
export default nextConfig;
```

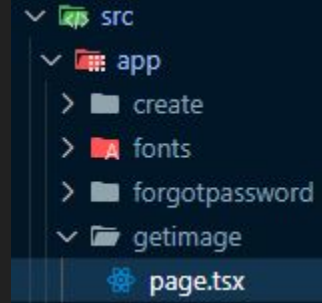


Agora vamos fazer uma lista com as imagens

Criar uma pasta getimage dentro do app,

Criar page.tsx

código no próximo slide para agilizar o processo



E agora para finalizar a segunda parte

Vamos colocar o website online via vercel!

Crie sua conta na vercel

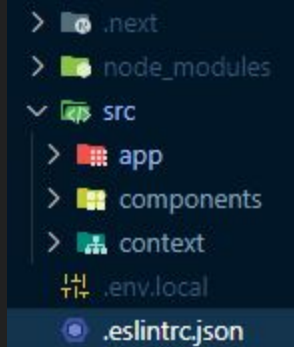
Conecte com o GitHub

Adicione o projeto em algum repositório seu no github

Adicione as variaveis de ambiente (.env.local) no local definido na Vercel

Adicione isto ao seu .eslintrc.json

```
{
  "extends": ["next/core-web-vitals", "next/typescript"],
  "rules": {
    "@typescript-eslint/no-explicit-any": "off",
    "react-hooks/exhaustive-deps": "off"
  }
}
```



AGRADECIMENTOS

Agradeço a todos!

segue link do github para referência

<https://github.com/viniciusgdoliveira/aula-firebase-nextjs>

segue link do projeto funcionando

<https://aula-firebase.vercel.app/>